

Contents

Módulo 5 - Gestión de errores y configuración avanzada	3
Punto 5.1 - Excepciones personalizadas	3
Objetivos de aprendizaje	3
Bloque 1 - Por qué excepciones personalizadas	3
T1.1 - El problema de las excepciones genéricas	3
T1.2 - Tipos de errores en una API REST	3
T1.3 - Excepciones checked vs unchecked	4
Bloque 2 - Jerarquía de excepciones	4
T2.1 - Una clase base común	4
T2.2 - Excepciones específicas	5
Bloque 3 - Información en las excepciones	6
T3.1 - Código de error interno	6
T3.2 - Datos adicionales	7
T3.3 - Trazabilidad con la causa	7
Bloque 4 - Lanzar excepciones desde el servicio	8
T4.1 - Dónde lanzar las excepciones	8
T4.2 - Lanzar con contexto	8
Bloque 5 - Buenas prácticas y tests	9
T5.1 - Buenas prácticas con excepciones	9
T5.3 - Tests negativos	10
Resumen de la teoría	10
Punto 5.2 - Respuestas de error estándar	11
Objetivos de aprendizaje	11
Bloque 1 - Por qué respuestas de error estándar	11
T1.1 - El problema de las respuestas inconsistentes	11
T1.2 - Ventajas de una estructura estándar	11
T1.3 - El estándar RFC 7807	12
Bloque 2 - Estructura de una respuesta de error	12
T2.1 - Campos básicos	12
T2.2 - Campos adicionales	13
T2.3 - Errores de validación	14
Bloque 3 - Construcción de la respuesta	14
T3.1 - Un DTO para el error	14
T3.3 - Un método de ayuda para construir errores	15
Bloque 4 - Errores de validación	16
T4.1 - MethodArgumentNotValidException	16
T4.2 - Otros tipos de errores de validación	17
T4.3 - Errores de parámetros	17
Bloque 5 - Buenas prácticas y tests	18
T5.1 - Buenas prácticas con respuestas de error	18
T5.2 - Tests para respuestas de error	18
Resumen de la teoría	19
Punto 5.3 - Manejo global de excepciones	19
Objetivos de aprendizaje	19
Bloque 1 - Qué es @RestControllerAdvice	20
T1.1 - Definición y origen	20
T1.2 - @ControllerAdvice vs @RestControllerAdvice	20
T1.3 - Cómo funciona internamente	20
Bloque 2 - Estructura del manejador global	21
T2.1 - Un manejador por tipo de excepción	21
T2.2 - Jerarquía de excepciones y especificidad	21
T2.3 - Orden con @Order	22

Bloque 3 - Aplicación selectiva	22
T3.1 - Aplicar a paquetes concretos	22
T3.2 - Combinación de manejadores locales y globales	23
T3.3 - Prioridad y orden de búsqueda	23
Bloque 4 - Casos especiales	24
T4.1 - Manejo de excepciones de validación	24
T4.2 - Manejo de excepciones de seguridad	24
T4.3 - Manejo de excepciones técnicas	24
Bloque 5 - Buenas prácticas y tests	25
T5.1 - Buenas prácticas con @RestControllerAdvice	25
T5.3 - Tests para excepciones no controladas	26
Resumen de la teoría	26
Punto 5.4 - Configuración básica de perfiles y properties	27
Objetivos de aprendizaje	27
Bloque 1 - Configuración externalizada	27
T1.1 - Qué es application.properties y cómo se carga	27
T1.2 - application.properties vs application.yml	27
T1.3 - Propiedades de Spring Boot y propiedades personalizadas	28
Bloque 2 - Perfiles de Spring Boot	28
T2.1 - Qué son los perfiles y por qué existen	28
T2.2 - application-{perfil}.properties	29
T2.3 - Cómo activar perfiles	29
Bloque 3 - @Profile y beans condicionales	30
T3.1 - @Profile en clases @Component/@Service	30
T3.2 - @Profile en métodos @Bean	30
T3.3 - Beans por defecto y beans específicos	31
Bloque 4 - @ConfigurationProperties	31
T4.1 - Qué es @ConfigurationProperties	31
T4.2 - Estructuras anidadas	32
Bloque 5 - Orden de prioridad y buenas prácticas	34
T5.1 - Orden de prioridad de las propiedades	34
T5.2 - Secretos y variables de entorno	34
T5.3 - Buenas prácticas con perfiles y properties	34
Resumen de la teoría	35
Punto 5.5 - Configuración de CORS para consumo desde front-end	35
Objetivos de aprendizaje	35
Bloque 1 - La política de mismo origen	35
T1.1 - Qué es el origen de una página web	35
Bloque 2 - Peticiones simples y preflight	37
T2.1 - Peticiones simples	37
T2.2 - Peticiones preflight	37
T2.3 - Cuándo se produce el preflight	37
Bloque 3 - Cómo configurar CORS en Spring Boot	38
T3.1 - Configuración global con WebMvcConfigurer	38
T3.2 - Configuración a nivel de controlador	38
T3.3 - Nota sobre Spring Security	39
Bloque 4 - Casos prácticos	39
T4.1 - Front-end en desarrollo	39
T4.2 - Front-end en producción	40
T4.3 - Diferencias entre desarrollo y producción	40
Bloque 5 - Errores comunes y buenas prácticas	41
T5.1 - Errores comunes	41
T5.2 - Buenas prácticas	42
T5.3 - Diagnóstico de errores CORS	42
Resumen de la teoría	42

Módulo 5 - Gestión de errores y configuración avanzada

Baseline: Java 17, Spring Boot 3.5.16 y Maven Wrapper 3.9.16. M5 evoluciona el proyecto acumulativo cerrado en M4-R1 sin perder persistencia, endpoints ni tests heredados.

Actualización editorial: la fuente presenta RFC 7807. Se conserva y explica su modelo, pero se indica que RFC 9457 lo sustituyó como especificación vigente de Problem Details.

Punto 5.1 - Excepciones personalizadas

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar por qué las excepciones personalizadas son necesarias en una API REST.
2. Diferenciar excepciones de negocio, de validación, de recursos y técnicas.
3. Diseñar una jerarquía de excepciones coherente para un proyecto.
4. Implementar excepciones personalizadas con información estructurada.
5. Lanzar excepciones personalizadas desde el servicio con contexto.
6. Escribir tests para verificar que las excepciones se lanzan correctamente.
7. Diagnosticar y resolver los errores más comunes al trabajar con excepciones.

Bloque 1 - Por qué excepciones personalizadas

T1.1 - El problema de las excepciones genéricas

Cuando algo va mal en un servicio, la tentación es lanzar una excepción genérica. Por ejemplo:

```
if (repositorio.existsByDni(dni)) {  
    throw new RuntimeException("Ya existe un alumno con ese DNI");  
}
```

Funciona: el programa se detiene y el mensaje se propaga. Pero tiene varios problemas:

- No se puede distinguir el tipo de error. ¿Es un duplicado? ¿Es un recurso no encontrado? ¿Es un error de validación? Con `RuntimeException`, todos son iguales. El manejador de errores no puede decidir qué código HTTP devolver.
- No lleva información estructurada. El mensaje es un string. No se puede acceder a los campos concretos que fallaron (el DNI duplicado, el curso no encontrado...). El cliente recibe un texto, no un objeto.
- No se puede capturar selectivamente. Si quieres capturar solo los errores de “recurso no encontrado”, tendrías que comprobar el mensaje, lo cual es frágil.
- No expresa la intención del dominio. “`RuntimeException`” no dice nada sobre el problema. “`AlumnoNoEncontradoException`” sí.

Las excepciones personalizadas resuelven estos problemas: cada tipo de error tiene su propia clase, con su propia información, y se puede manejar de forma distinta.

T1.2 - Tipos de errores en una API REST

En una API REST, los errores se agrupan en varias categorías, cada una con su propio código HTTP:

Errores de validación (400 Bad Request): los datos de entrada no cumplen el formato esperado. Un campo obligatorio vacío, un DNI con longitud incorrecta, un email mal formado. Se detectan antes de tocar la lógica de negocio.

Errores de recurso no encontrado (404 Not Found): el recurso solicitado no existe. Un alumno con un ID que no está en la base de datos, un curso que se ha eliminado.

Errores de conflicto (409 Conflict): la operación no se puede completar porque entra en conflicto con el estado actual. Un DNI duplicado, un curso con alumnos que no se puede eliminar, una matrícula ya existente. Errores de autenticación (401 Unauthorized): el cliente no se ha autenticado. Falta el token o es inválido.

Errores de autorización (403 Forbidden): el cliente está autenticado pero no tiene permisos para esa operación.

Errores técnicos (500 Internal Server Error): algo ha fallado en el servidor. Una conexión a base de datos perdida, un fichero no encontrado, un fallo en un servicio externo.

Cada categoría se corresponde con un tipo de excepción. Tener una jerarquía clara permite que el manejador global de errores decida qué código HTTP devolver sin tener que inspeccionar mensajes.

T1.3 - Excepciones checked vs unchecked

En Java, hay dos tipos de excepciones:

Checked: extienden Exception (pero no RuntimeException). El compilador obliga a capturarlas o declararlas con throws. Ejemplos: IOException, SQLException.

Unchecked: extienden RuntimeException. El compilador no obliga a capturarlas. Ejemplos: NullPointerException, IllegalArgumentException. Para las excepciones de negocio, la recomendación es usar unchecked. Las razones son varias:

- No contaminan las firmas de los métodos. Si crearAlumno pudiera lanzar AlumnoDuplicadoException, no quieres que todos los métodos que lo llaman tengan que declararlo con throws. Con unchecked, no hace falta.
- Rollback automático en Spring. Spring hace rollback por defecto con RuntimeException. Con checked, no lo hace salvo que se use rollbackFor. Usar unchecked evita tener que configurarlo.
- Semántica clara. Una violación de regla de negocio no es algo de lo que el llamador pueda recuperarse. Es un error que se propaga hasta el manejador global.

Por tanto, todas nuestras excepciones personalizadas extenderán RuntimeException (o una clase base que extienda RuntimeException).

Pregunta de reflexión: ¿Por qué las excepciones de negocio se modelan como unchecked y no como checked?

Bloque 2 - Jerarquía de excepciones

T2.1 - Una clase base común

Antes de crear excepciones específicas, conviene tener una clase base que agrupe todas las excepciones de negocio del proyecto. Esa clase base permite: - Capturar todas las excepciones de negocio con un solo catch. Útil para el manejador global. - Añadir información común. Por ejemplo, un código de error interno, un campo adicional, una marca de "recurso afectado". - Diferenciar las excepciones propias de las de terceros. Si una excepción no extiende la clase base, es que viene de fuera.

Un ejemplo de clase base:

```
public abstract class AplicacionException extends RuntimeException {  
  
    private final String codigo;  
  
    protected AplicacionException(String codigo, String mensaje) {  
        super(mensaje);  
        this.codigo = codigo;  
    }  
  
    protected AplicacionException(String codigo, String mensaje, Throwable causa) {
```

```

    super(mensaje, causa);
    this.codigo = codigo;
}

public String getCodigo() {
    return codigo;
}
}

```

abstract porque no se instancia directamente: es la base de otras excepciones.

codigo es un identificador interno del error. Por ejemplo, ALUMNO_NO_ENCONTRADO, DNI_DUPLICADO. Es útil para que el cliente pueda reaccionar sin depender del mensaje (que puede cambiar o traducirse).

El constructor recibe el código y el mensaje, y opcionalmente una causa. La causa es útil cuando la excepción envuelve a otra (por ejemplo, un fallo de base de datos).

T2.2 - Excepciones específicas

Sobre la clase base, se crean las excepciones específicas. Cada una representa un tipo de error concreto:

RecursoNoEncontradoException: el recurso no existe. Se traduce a 404. Puede llevar el tipo de recurso y el ID.

```

public class RecursoNoEncontradoException extends AplicacionException {

    private final String recurso;
    private final Object id;

    public RecursoNoEncontradoException(String recurso, Object id) {
        super("RECURSO_NO_ENCONTRADO",
            String.format("No se encontró %s con id %s", recurso, id));
        this.recurso = recurso;
        this.id = id;
    }

    public String getRecurso() { return recurso; }
    public Object getId() { return id; }
}

```

RecursoDuplicadoException: ya existe un recurso con los mismos datos únicos. Se traduce a 409. Puede llevar el campo duplicado y el valor.

```

public class RecursoDuplicadoException extends AplicacionException {

    private final String campo;
    private final Object valor;

    public RecursoDuplicadoException(String recurso, String campo, Object valor) {
        super("RECURSO_DUPLICADO",
            String.format("Ya existe %s con %s = %s", recurso, campo, valor));
        this.campo = campo;
        this.valor = valor;
    }

    public String getCampo() { return campo; }
    public Object getValor() { return valor; }
}

```

OperacionNoPermitidaException: la operación no se puede completar por el estado actual. Se traduce a 409. Puede llevar el motivo.

```
public class OperacionNoPermitidaException extends AplicacionException {

    public OperacionNoPermitidaException(String mensaje) {
        super("OPERACION_NO_PERMITIDA", mensaje);
    }
}
```

ErrorTecnicoException: un fallo técnico (base de datos, servicio externo). Se traduce a 500. Puede llevar la causa original.

```
public class ErrorTecnicoException extends AplicacionException {

    public ErrorTecnicoException(String mensaje, Throwable causa) {
        super("ERROR_TECNICO", mensaje, causa);
    }
}
```

Cada excepción tiene su propio propósito y lleva la información que el manejador necesita para construir la respuesta de error. ### T2.3 - Cuándo crear una nueva excepción

No todas las situaciones necesitan una excepción nueva. La regla es: crear una excepción nueva cuando el manejador deba hacer algo distinto.

Si dos errores se traducen al mismo código HTTP y llevan la misma información, pueden compartir excepción. Si se traducen a códigos distintos o llevan información distinta, conviene separarlas.

Ejemplos:

- Alumno no encontrado y Curso no encontrado → ambos 404. Pueden usar RecursoNoEncontradoException con el tipo de recurso como parámetro. No hace falta una excepción por cada entidad.
- DNI duplicado y Nombre de curso duplicado → ambos 409. Pueden usar RecursoDuplicadoException. No hace falta una por campo.
- Eliminar un curso con alumnos → 409, pero con un mensaje específico. Puede usar OperacionNoPermitidaException.
- Fallo al conectar con Hacienda → 500, con la causa técnica. ErrorTecnicoException.

La clave es no sobre-diseñar. Si tienes 50 excepciones, es que has ido demasiado lejos. Con 5-10 excepciones bien pensadas se cubren la mayoría de los casos. > **Pregunta de reflexión:** ¿Cuándo crearías una excepción específica para “alumno no encontrado” en lugar de usar una genérica de “recurso no encontrado”?

Bloque 3 - Información en las excepciones

T3.1 - Código de error interno

El código de error interno es un string que identifica el tipo de error de forma estable. No cambia aunque el mensaje sí.

Ventajas:

- El cliente puede reaccionar sin depender del mensaje. Si el mensaje se traduce al inglés, el código sigue siendo DNI_DUPLICADO.
- Es estable entre versiones. El mensaje puede mejorar; el código se mantiene.
- Es fácil de documentar. En la documentación de la API se listan los códigos posibles.

Convención de nombres:

- UPPER_SNAKE_CASE: DNI_DUPLICADO, RECURSO_NO_ENCONTRADO.

- En inglés o español, pero consistente. En proyectos internacionales, inglés. En proyectos nacionales, español.
- Específico pero no demasiado. ALUMNO_DNI_DUPLICADO es mejor que ERROR o ERROR_1.

En el manejador global, el código se incluye en la respuesta JSON:

```
{
  "timestamp": "2025-01-15T10:00:00Z",
  "status": 409,
  "codigo": "RECURSO_DUPLICADO",
  "mensaje": "Ya existe Alumno con dni = 12345678A",
  "path": "/api/v1/alumnos"
}
```

T3.2 - Datos adicionales

Además del código y el mensaje, una excepción puede llevar datos adicionales que ayuden al cliente a entender qué ha fallado:

Para RecursoNoEncontradoException:

- El tipo de recurso: "Alumno", "Curso", "Expediente".
- El ID que no se encontró: 12345. Para RecursoDuplicadoException:
- El campo que causó el conflicto: "dni", "nombre", "email".
- El valor duplicado: "12345678A".

Para OperacionNoPermitidaException:

- El motivo: "El curso tiene 5 alumnos asignados".
- El recurso afectado: "Curso" con ID 1.

Para ErrorTecnicoException:

- La causa original (una excepción de otro sistema).
- El servicio externo que ha fallado: "Hacienda", "SeguridadSocial".

Estos datos se incluyen en la respuesta JSON, en un campo adicional:

```
{
  "timestamp": "2025-01-15T10:00:00Z",
  "status": 409,
  "codigo": "RECURSO_DUPLICADO",
  "mensaje": "Ya existe Alumno con dni = 12345678A",
  "detalles": {
    "campo": "dni",
    "valor": "12345678A"
  },
  "path": "/api/v1/alumnos"
}
```

El cliente puede leer detalles.campo y detalles.valor para mostrar un mensaje más específico o para corregir el formulario.

T3.3 - Trazabilidad con la causa

Cuando una excepción de negocio envuelve una excepción técnica, conviene guardar la causa original. Es útil para depurar.

Ejemplo:

```
try {
    respuesta = clienteHacienda.consultar(dni);
} catch (IOException e) {
    throw new ErrorTecnicoException(
        "Error al consultar Hacienda para el DNI " + dni, e);
}
```

El segundo argumento del constructor es la causa original. La excepción de negocio (ErrorTecnicoException) envuelve a la técnica (IOException). El manejador puede registrar ambas en los logs:

```
log.error("Error técnico: {} - Causa: {}",
    ex.getMessage(), ex.getCause().getMessage(), ex);
```

La causa se registra en los logs, pero no se expone al cliente. El cliente recibe un mensaje genérico ("Error interno del servidor") y un código de error. Los detalles técnicos (traza de pila, nombres de clases) se quedan en el servidor.

Esta separación es importante: el cliente no necesita saber que la base de datos ha lanzado un SQLException con código 23505. Solo necesita saber que ha habido un error y que se ha registrado.

Pregunta de reflexión: ¿Por qué no se debe exponer la traza de pila al cliente? ¿Qué información podría filtrar?

Bloque 4 - Lanzar excepciones desde el servicio

T4.1 - Dónde lanzar las excepciones

Las excepciones se lanzan desde el servicio, no desde el controlador ni desde el repositorio. El controlador no debe lanzar excepciones de negocio. Su trabajo es recibir la petición y delegar en el servicio. Si el servicio lanza una excepción, el controlador la deja propagar.

El repositorio no debe lanzar excepciones de negocio. Su trabajo es acceder a datos. Si algo falla, propaga la excepción técnica que le llegue de la base de datos. El servicio la envuelve en una excepción de negocio si hace falta.

El servicio es el único que conoce las reglas de negocio. Es el único que sabe si un DNI está duplicado, si un curso tiene alumnos, si un alumno no existe. Por eso lanza las excepciones de negocio.

Regla: las excepciones de negocio se lanzan desde el servicio.

T4.2 - Lanzar con contexto

Cuando lanzas una excepción, incluye toda la información que el manejador pueda necesitar:

```
public AlumnoDTO crear(AlumnoRequestDTO request) {
    if (repositorio.existsByDni(request.getDni())) {
        throw new RecursoDuplicadoException("Alumno", "dni", request.getDni());
    }
    // ...
}
```

En este ejemplo, la excepción lleva:

- El tipo de recurso: "Alumno".
- El campo: "dni".
- El valor: request.getDni().

Con esa información, el manejador puede construir una respuesta como:

```
{
    "codigo": "RECURSO_DUPLICADO",
```

```

    "mensaje": "Ya existe Alumno con dni = 12345678A",
    "detalles": {
        "campo": "dni",
        "valor": "12345678A"
    }
}

```

Es mucho más útil que un simple “Ya existe un alumno con ese DNI”. ### T4.3 - Envolver excepciones técnicas

Cuando una excepción técnica se lanza desde el repositorio o desde un servicio externo, el servicio puede envolverla en una excepción de negocio más significativa:

```

@Transactional
public AlumnoDTO crear(AlumnoRequestDTO request) {
    try {
        // ...
        return toDTO(repositorio.save(entidad));
    } catch (DataIntegrityViolationException e) {
        throw new RecursoDuplicadoException("Alumno", "dni", request.getDni());
    }
}

```

DataIntegrityViolationException es una excepción de Spring que se lanza cuando la base de datos detecta una violación de constraint. El servicio la captura y lanza una excepción de negocio más significativa.

Importante: no captures Exception genérica. Captura solo las excepciones que sabes manejar. Las demás se propagan.

No abusos de try-catch. Solo envuelve las excepciones cuando aportes información adicional. Si no, deja que se propaguen.

Pregunta de reflexión: ¿Por qué el servicio debe envolver las excepciones técnicas en excepciones de negocio?

Bloque 5 - Buenas prácticas y tests

T5.1 - Buenas prácticas con excepciones

Extender RuntimeException (o una base que extienda RuntimeException). No usar Exception checked.

Usar una clase base común. AplicacionException agrupa todas las excepciones del proyecto.

Incluir un código de error estable. RECURSO_NO_ENCONTRADO, DNI_DUPLICADO. No cambia aunque el mensaje sí.

Incluir datos adicionales. El campo que falla, el valor, el recurso, el ID.

Guardar la causa original cuando se envuelve una excepción técnica. Útil para depurar.

No exponer detalles técnicos al cliente. El mensaje debe ser claro para el usuario, no para el desarrollador.

No crear excepciones innecesarias. Si dos errores se traducen al mismo código HTTP y llevan la misma información, comparten excepción.

Documentar las excepciones en la API. En OpenAPI, describir los códigos de error posibles. ### T5.2 - Tests para excepciones

Un buen test de excepciones verifica:

- Que se lanza la excepción correcta cuando se cumple la condición.
- Que el mensaje es el esperado.

- Que los datos adicionales son los correctos.
- Que no se lanza cuando no se cumple la condición.

Ejemplo:

```
@Test
void crear_debeLanzarRecursoDuplicado_cuandoDniExiste() {
    when(repositorio.existsByDni("12345678A")).thenReturn(true);
    AlumnoRequestDTO request = new AlumnoRequestDTO();
    request.setDni("12345678A");

    RecursoDuplicadoException ex = assertThrows(
        RecursoDuplicadoException.class,
        () -> service.crear(request)
    );
    assertEquals("dni", ex.getCampo());
    assertEquals("12345678A", ex.getValor());
    assertEquals("RECURSO_DUPLICADO", ex.getCodigo());
}
```

El test verifica que se lanza la excepción correcta, y que los datos adicionales (campo, valor, código) son los esperados.

T5.3 - Tests negativos

Tan importante como probar que se lanza la excepción es probar que no se lanza cuando no debe:

```
@Test
void crear_noDebeLanzarExcepcion_cuandoDniNoExiste() {
    when(repositorio.existsByDni("12345678A")).thenReturn(false);
    when(repositorio.save(any())).thenAnswer(inv -> inv.getArgument(0));
    AlumnoRequestDTO request = new AlumnoRequestDTO();
    request.setDni("12345678A");

    assertDoesNotThrow(() -> service.crear(request));
}
```

El test verifica que, cuando el DNI no existe, no se lanza excepción. Si alguien cambia la lógica y lanza la excepción siempre, el test falla.

Los tests negativos protegen contra regresiones: cambios que rompen el comportamiento esperado sin que te des cuenta.

Pregunta de reflexión: ¿Por qué es importante probar que NO se lanza una excepción?
¿Qué tipo de error detecta?

Resumen de la teoría

- Excepciones personalizadas: cada tipo de error tiene su clase.
- Tipos de error: validación (400), no encontrado (404), conflicto (409), autenticación (401), autorización (403), técnico (500).
- Checked vs unchecked: las de negocio son unchecked (RuntimeException).
- Clase base: AplicacionException agrupa todas las del proyecto.
- Excepciones específicas: RecursoNoEncontradoException, RecursoDuplicadoException, OperacionNoPermitidaException, ErrorTecnicoException.
- Código de error interno: RECURSO_NO_ENCONTRADO, DNI_DUPLICADO.
- Datos adicionales: el campo, el valor, el recurso, el ID.
- Causa original: para depurar, no se expone al cliente.
- Dónde lanzar: desde el servicio.
- Envolver excepciones técnicas: para darles significado de negocio.

- Buenas prácticas: clase base, código estable, datos adicionales, no exponer detalles.
- Tests: verificar que se lanza y que no se lanza.

Punto 5.2 - Respuestas de error estándar

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar por qué las respuestas de error deben tener una estructura consistente.
2. Diseñar un DTO de error estándar con todos los campos útiles.
3. Conocer el estándar RFC 7807 (Problem Details for HTTP APIs).
4. Diferenciar respuestas de error simples de respuestas con múltiples errores de validación.
5. Construir respuestas de error que el cliente pueda procesar programáticamente.
6. Escribir tests para verificar la estructura de las respuestas de error.
7. Diagnosticar y resolver los errores más comunes al construir respuestas de error.

Bloque 1 - Por qué respuestas de error estándar

T1.1 - El problema de las respuestas inconsistentes

Cuando una API no tiene un formato estándar para los errores, cada endpoint devuelve el error de una forma distinta. Unos devuelven un string, otros un objeto con campos distintos, otros un array. El cliente que consume la API tiene que escribir código específico para cada caso, y cualquier cambio en un endpoint rompe su lógica de manejo de errores.

Imagina que tienes tres endpoints. El primero devuelve un error así:

```
{"error": "Not found"}
```

El segundo, así:

```
{"status": 404, "mensaje": "Alumno no encontrado", "recurso": "Alumno", "id": 123}
```

Y el tercero, así:

```
{
  "codigo": 404,
  "descripcion": "Recurso no encontrado",
  "detalles": "El alumno con id 123 no existe"
}
```

El cliente tiene que comprobar si la respuesta tiene el campo error, mensaje, descripcion o ninguno de los tres. Es un caos. Cada endpoint es un mundo. La solución es definir una estructura de error estándar que todos los endpoints usen. El cliente escribe un solo manejador de errores y funciona con todos. Es más fácil para el cliente, más fácil de documentar, y más fácil de evolucionar.

T1.2 - Ventajas de una estructura estándar

Una estructura de error estándar aporta varias ventajas:

Consistencia para el cliente. Todos los errores tienen la misma forma. El cliente escribe un solo manejador de errores. No tiene que comprobar la forma de cada respuesta.

Información estructurada. El cliente puede acceder a los campos del error de forma programática. No tiene que parsear strings. Puede leer codigo para decidir qué hacer, campo para resaltar el campo del formulario, detalles para mostrar información adicional.

Facilidad de documentación. La documentación de la API describe una sola estructura de error. El cliente sabe qué esperar. No hay que documentar cada endpoint por separado.

Trazabilidad. Un campo timestamp permite al cliente reportar el error con el momento exacto. Un campo path indica qué URL falló. Un campo traceId permite al servidor localizar el error en los logs. Internacionalización. Un campo codigo estable permite al cliente traducir el mensaje al idioma del usuario. El mensaje del servidor puede estar en español, pero el cliente puede mostrar un mensaje en inglés o catalán según el código.

Evolución controlada. Si se añade un nuevo campo al error (por ejemplo, sugerencia), los clientes existentes no fallan. Simplemente ignoran el campo que no conocen.

T1.3 - El estándar RFC 7807

En 2016, la IETF publicó el RFC 7807 - Problem Details for HTTP APIs. Es un estándar que define una estructura común para los errores de APIs HTTP. No es obligatorio, pero muchas APIs lo siguen porque es un estándar reconocido.

La estructura del RFC 7807 es:

```
{
  "type": "https://example.com/errores/recurso-no-encontrado",
  "title": "Recurso no encontrado",
  "status": 404,
  "detail": "No se encontró el alumno con id 12345",
  "instance": "/api/v1/alumnos/12345"
}
```

Los campos son:

- type: una URI que identifica el tipo de error. Puede ser una URL a la documentación del error.
- title: un título legible del tipo de error.
- status: el código HTTP.
- detail: una descripción específica del error concreto.
- instance: la URI del recurso que ha provocado el error.

El RFC 7807 permite extensiones: campos adicionales según las necesidades. Por ejemplo, para errores de validación:

```
{
  "type": "https://example.com/errores/validacion",
  "title": "Error de validación",
  "status": 400,
  "detail": "La petición tiene errores de validación",
  "instance": "/api/v1/alumnos",
  "errors": [
    {"field": "nombre", "message": "El nombre es obligatorio"},
    {"field": "dni", "message": "El DNI debe tener 9 caracteres"}
  ]
}
```

Spring Boot 3 incluye soporte para RFC 7807 a través de la clase ProblemDetail. Se puede activar con spring.mvc.problemdetails.enabled=true. Pero también se puede implementar una estructura propia, que es lo que haremos en este curso, porque da más control.

Pregunta de reflexión: ¿Qué ventaja tiene seguir un estándar como RFC 7807? ¿Qué desventaja tiene frente a una estructura propia?

Bloque 2 - Estructura de una respuesta de error

T2.1 - Campos básicos

Una respuesta de error estándar debe tener, como mínimo, estos campos:

timestamp: el momento en que se produjo el error, en formato ISO 8601. Permite al cliente y al servidor correlacionar logs.

status: el código HTTP. Es redundante con la línea de estado, pero facilita el parseo del cuerpo sin tener que mirar las cabeceras.

codigo: un identificador interno del error. RECURSO_NO_ENCONTRADO, DNI_DUPLICADO. Es-table, no cambia aunque el mensaje sí.

mensaje: una descripción legible del error. Orientada al usuario final o al desarrollador que depura.

path: la URL que provocó el error. Útil para reproducirlo. Un ejemplo mínimo:

```
{
  "timestamp": "2025-01-15T10:00:00Z",
  "status": 404,
  "codigo": "RECURSO_NO_ENCONTRADO",
  "mensaje": "No se encontró Alumno con id 12345",
  "path": "/api/v1/alumnos/12345"
}
```

Estos cinco campos son el mínimo. Con ellos, el cliente sabe qué ha pasado, cuándo, dónde y con qué código.

T2.2 - Campos adicionales

Además de los básicos, una respuesta de error puede incluir campos adicionales según el tipo de error:

detalles: un objeto con información específica del error. Por ejemplo, para un recurso duplicado, el campo y el valor:

```
{
  "detalles": {
    "campo": "dni",
    "valor": "12345678A"
  }
}
```

errors: una lista de errores, útil para errores de validación con múltiples campos:

```
{
  "errors": [
    {"field": "nombre", "message": "El nombre es obligatorio"},
    {"field": "dni", "message": "El DNI debe tener 9 caracteres"}
  ]
}
```

sugerencia: una sugerencia para el cliente sobre cómo resolver el error:

```
{
  "sugerencia": "Compruebe que el DNI tiene 9 caracteres y vuelva a intentarlo"
}
```

documentacion: una URL a la documentación del error:

```
{
  "documentacion": "https://api.mecd.es/errores/RECURSO_NO_ENCONTRADO"
}
```

traceId: un identificador único del error, que permite al servidor localizarlo en los logs:

```
{
  "traceId": "a1b2c3d4-e5f6-7890-1234-567890abcdef"
}
```

No todos los errores necesitan todos los campos. Los básicos siempre; los adicionales, cuando aporten valor.

T2.3 - Errores de validación

Los errores de validación son un caso especial. Cuando el cliente envía un formulario con varios campos inválidos, no es útil devolver un solo error: el cliente quiere saber todos los errores de una vez, para que el usuario los corrija todos juntos.

La estructura típica para errores de validación incluye un array errors con un objeto por cada campo inválido:

```
{
  "timestamp": "2025-01-15T10:00:00Z",
  "status": 400,
  "codigo": "VALIDACION",
  "mensaje": "La petición tiene errores de validación",
  "path": "/api/v1/alumnos",
  "errors": [
    {
      "field": "nombre",
      "rejectedValue": "",
      "message": "El nombre es obligatorio"
    },
    {
      "field": "dni",
      "rejectedValue": "123",
      "message": "El DNI debe tener 9 caracteres"
    },
    {
      "field": "fechaNacimiento",
      "rejectedValue": null,
      "message": "La fecha de nacimiento es obligatoria"
    }
  ]
}
```

Cada objeto del array incluye: - field: el nombre del campo que falló. - rejectedValue: el valor que se rechazó. Útil para que el cliente sepa qué envió. - message: el mensaje de error.

Con esta estructura, el cliente puede recorrer el array y mostrar todos los errores juntos, o resaltar los campos correspondientes.

Pregunta de reflexión: ¿Por qué es mejor devolver todos los errores de validación juntos en lugar de uno por uno?

Bloque 3 - Construcción de la respuesta

T3.1 - Un DTO para el error

La forma más limpia de construir una respuesta de error es con un DTO específico. Se crea una clase ErrorResponse con los campos que queramos exponer:

```
public class ErrorResponse {

    private String timestamp;
```

```

    private int status;
    private String codigo;
    private String mensaje;
    private String path;
    private Map<String, Object> detalles;
    private List<ValidationError> errors;

    // constructores, getters y setters
}

```

Y un DTO para los errores de validación:

```

public class ValidationError {

    private String field;
    private Object rejectedValue;
    private String message;

    // constructores, getters y setters
}

```

Con estos DTOs, el manejador construye el objeto y lo devuelve. Spring lo serializa a JSON con Jackson. Se pueden añadir anotaciones como @JsonInclude(NON_NULL) para omitir los campos nulos.

Ventaja: la estructura del error está tipada. El compilador la verifica. El IDE autocompleta. Los tests pueden verificar los campos. **### T3.2 - Construir la respuesta con ResponseEntity**

El manejador construye la respuesta con ResponseEntity:

```

@ExceptionHandler(RecursoNoEncontradoException.class)
public ResponseEntity<ErrorResponse> handleNoEncontrado(
    RecursoNoEncontradoException ex,
    HttpServletRequest request) {
    ErrorResponse error = new ErrorResponse();
    error.setTimestamp(Instant.now().toString());
    error.setStatus(HttpStatus.NOT_FOUND.value());
    error.setCodigo(ex.getCodigo());
    error.setMensaje(ex.getMessage());
    error.setPath(request.getRequestURI());
    error.setDetalles(Map.of(
        "recurso", ex.getRecurso(),
        "id", ex.getId()
    ));
    return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
}

```

HttpServletRequest request se inyecta automáticamente. Se usa para obtener la URI (request.getRequestURI()) y, si se quiere, otros

datos como las cabeceras o el método HTTP.

Instant.now().toString() genera el timestamp en formato ISO 8601.

Map.of(...) construye el objeto de detalles.

ResponseEntity.status(...).body(error) construye la respuesta con el código y el cuerpo.

T3.3 - Un método de ayuda para construir errores

Para no repetir el código de construcción en cada manejador, se puede extraer a un método privado:

```
private ResponseEntity<ErrorResponse> buildErrorResponse(
    HttpStatus status,
    String codigo,
    String mensaje,
    HttpServletRequest request,
    Map<String, Object> detalles) {
    ErrorResponse error = new ErrorResponse();
    error.setTimestamp(Instant.now().toString());
    error.setStatus(status.value());
    error.setCodigo(codigo);
    error.setMensaje(mensaje);
    error.setPath(request.getRequestURI());
    error.setDetalles(detalles);
    return ResponseEntity.status(status).body(error);
}
```

Cada manejador llama a este método con sus parámetros:

```
@ExceptionHandler(RecursoNoEncontradoException.class)
public ResponseEntity<ErrorResponse> handleNoEncontrado(
    RecursoNoEncontradoException ex,
    HttpServletRequest request) {
    return buildErrorResponse(
        HttpStatus.NOT_FOUND,
        ex.getCodigo(),
        ex.getMessage(),
        request,
        Map.of("recurso", ex.getRecurso(), "id", ex.getId())
    );
}
```

Ventaja: el código se repite menos. Si hay que cambiar la estructura del error (añadir un campo), se cambia en un solo sitio.

Pregunta de reflexión: ¿Por qué es mejor tener un método de ayuda que construir la respuesta en cada manejador?

Bloque 4 - Errores de validación

T4.1 - MethodArgumentNotValidException

Ya conocemos MethodArgumentNotValidException del Módulo 3. Es la excepción que Spring MVC lanza cuando @Valid detecta errores de validación en el cuerpo de la petición.

Contiene un BindingResult con la lista de errores de campo. Cada error es un FieldError con:

- getField(): el nombre del campo que falló.
- getRejectedValue(): el valor que se rechazó.
- getDefaultMessage(): el mensaje de validación.

Para construir la respuesta estándar, se recorren los FieldError y se crea un ValidationError por cada uno:

```
List<ValidationError> errores = ex.getBindingResult()
    .getFieldErrors()
    .stream()
    .map(fe -> new ValidationError(
        fe.getField(),
        fe.getRejectedValue(),
        fe.getDefaultMessage()))
    .toList();
```

Y se añaden al ErrorResponse:

```
error.setErrorserrores);
```

T4.2 - Otros tipos de errores de validación

Además de MethodArgumentNotValidException, hay otras excepciones de validación que conviene manejar:

ConstraintViolationException: se lanza cuando se validan parámetros de método (@RequestParam, @PathVariable) con anotaciones de Bean Validation. Por ejemplo, si un @PathVariable tiene @Min(1) y llega un 0.

HttpMessageNotReadableException: se lanza cuando el cuerpo de la petición no se puede leer. Por ejemplo, si el JSON está mal formado o el Content-Type no es el correcto. MethodArgumentTypeMismatchException: se lanza cuando un parámetro no se puede convertir al tipo esperado. Por ejemplo, si @PathVariable Long id recibe "abc".

MissingServletRequestParameterException: se lanza cuando falta un @RequestParam obligatorio.

Cada una de estas excepciones se puede manejar en el manejador global y traducir a una respuesta estándar de error.

Por ejemplo, para HttpMessageNotReadableException:

```
@ExceptionHandler(HttpMessageNotReadableException.class)
public ResponseEntity<ErrorResponse> handleMensajeNoLeible(
    HttpMessageNotReadableException ex,
    HttpServletRequest request) {
    return buildErrorResponse(
        HttpStatus.BAD_REQUEST,
        "MENSAJE_NO_LEIBLE",
        "El cuerpo de la petición no se puede leer",
        request,
        Map.of("detalle", ex.getMessage())
    );
}
```

T4.3 - Errores de parámetros

Los parámetros de la URL también pueden fallar. Por ejemplo:

Parámetro obligatorio ausente:

```
@GetMapping
public List<AlumnoDTO> listar(@RequestParam String curso) {
    // ...
}
```

Si el cliente no envía curso, Spring lanza MissingServletRequestParameterException.

Tipo de parámetro incorrecto:

```
@GetMapping("/{id}")
public ResponseEntity<AlumnoDTO> consultar(@PathVariable Long id) {
    // ...
}
```

Si el cliente envía /api/v1/alumnos/abc, Spring intenta convertir "abc" a Long y falla con MethodArgumentTypeMismatchException.

Validación de parámetro:

```

@GetMapping
public List<AlumnoDTO> listar(
    @RequestParam @Min(0) int page,
    @RequestParam @Min(1) @Max(100) int size) {
    // ...
}

```

Si page es negativo o size supera 100, Spring lanza `ConstraintViolationException`.

Todos estos errores se manejan en el manejador global con respuestas estándar.

Pregunta de reflexión: ¿Qué diferencia hay entre `MethodArgumentNotValidException` y `ConstraintViolationException`?

Bloque 5 - Buenas prácticas y tests

T5.1 - Buenas prácticas con respuestas de error

Estructura consistente. Todos los errores tienen la misma forma. El cliente escribe un solo manejador.

Códigos de error estables. `RECURSO_NO_ENCONTRADO`, `DNI_DUPLICADO`. No cambian aunque el mensaje sí. Mensajes claros pero sin detalles técnicos. “No se encontró el alumno con id 12345” es mejor que “`java.lang.NullPointerException` at line 42”. Los detalles técnicos van a los logs, no al cliente.

Timestamp en ISO 8601. Facilita el parseo y la correlación con logs.

Path siempre incluido. Ayuda al cliente a saber qué URL falló.

Errores de validación con lista completa. No devolver solo el primer error. Devolver todos.

No exponer la traza de pila. Nunca. Ni siquiera en desarrollo. Se registra en los logs.

No exponer nombres de clases ni paquetes. El cliente no necesita saber que el error viene de `es.mecd.demo.miproyecto.alumno.AlumnoService`.

Documentar los códigos de error. En OpenAPI, listar los códigos posibles por endpoint.

Idioma consistente. Todos los mensajes en el mismo idioma. Si el proyecto es multilingüe, usar códigos y traducir en el cliente.

T5.2 - Tests para respuestas de error

Un test de respuesta de error verifica: - El código de estado. - Los campos del cuerpo (timestamp, status, codigo, mensaje, path). - Los campos adicionales (detalles, errors). - La estructura general (que todos los campos estén presentes).

Ejemplo de test con `MockMvc`:

```

@Test
void consultar_debeDevolver404ConErrorEstandar_cuandoNoExiste() throws Exception {
    when(service.consultar(999L)).thenReturn(Optional.empty());

    mockMvc.perform(get("/api/v1/alumnos/999"))
        .andExpect(status().isNotFound())
        .andExpect(jsonPath("$.status").value(404))
        .andExpect(jsonPath("$.codigo").value("RECURSO_NO_ENCONTRADO"))
        .andExpect(jsonPath("$.mensaje").exists())
        .andExpect(jsonPath("$.path").value("/api/v1/alumnos/999"))
        .andExpect(jsonPath("$.timestamp").exists());
}

```

El test verifica que todos los campos están presentes y que los valores son correctos. ### T5.3 - Tests de errores de validación

Para errores de validación, el test verifica que el array errors está presente y que contiene los errores esperados:

```
@Test
void crear_debeDevolver400ConErroresDeValidacion() throws Exception {
    mockMvc.perform(post("/api/v1/alumnos")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
        {"nombre":"","apellidos":"López",
         "dni":"123", "fechaNacimiento":"2011-03-20",
         "curso":"4º"}
        """))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.codigo").value("VALIDACION"))
        .andExpect(jsonPath("$.errors").isArray())
        .andExpect(jsonPath("$.errors.length()").value(2))
        .andExpect(jsonPath("$.errors[?(@.field=='nombre')].message")
        .value("El nombre es obligatorio"))
        .andExpect(jsonPath("$.errors[?(@.field=='dni')].message")
        .value("El DNI debe tener 9 caracteres"));
}
```

El test verifica que hay dos errores, que uno es del campo nombre y otro del campo dni, y que los mensajes son los esperados. > **Pregunta de reflexión:** ¿Qué ventaja tiene que el test verifique la estructura del error y no solo el código de estado?

Resumen de la teoría

- Respuestas de error estándar: todos los errores tienen la misma forma.
- Ventajas: consistencia, información estructurada, documentación, trazabilidad, i18n, evolución.
- RFC 7807: estándar para errores de APIs HTTP. type, title, status, detail, instance.
- Campos básicos: timestamp, status, codigo, mensaje, path.
- Campos adicionales: detalles, errors, sugerencia, documentacion, traceId.
- Errores de validación: array errors con field, rejectedValue, message.
- DTOs: ErrorResponse y ValidationException.
- Construcción: ResponseEntity y método de ayuda.
- Excepciones de validación: MethodArgumentNotValidException, ConstraintViolationException, HttpResponseMessageNotReadableException, MethodArgumentTypeMismatchException.
- Buenas prácticas: consistencia, códigos estables, mensajes claros, no exponer traza.
- Tests: verificar código, campos y estructura.

Punto 5.3 - Manejo global de excepciones

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar qué es @RestControllerAdvice y cómo funciona internamente.
2. Diferenciar el manejo local (@ExceptionHandler en el controlador) del global.
3. Ordenar manejadores de excepciones con @Order cuando hay jerarquía.
4. Aplicar @RestControllerAdvice a paquetes o anotaciones concretas.
5. Manejar excepciones de validación, de seguridad y técnicas de forma centralizada.
6. Escribir tests que verifiquen el manejo global.

7. Diagnosticar y resolver los errores más comunes al configurar el manejo global.

Bloque 1 - Qué es @RestControllerAdvice

T1.1 - Definición y origen

@RestControllerAdvice es una anotación de Spring que marca una clase como manejador global de excepciones para todos los controladores de la aplicación. Es una especialización de @ControllerAdvice que añade @ResponseBody, lo que significa que los métodos devuelven el cuerpo de la respuesta directamente (JSON), no el nombre de una vista.

En Spring MVC, cuando un controlador lanza una excepción, el DispatcherServlet busca un manejador que sepa tratarla. Esa búsqueda se hace en dos niveles:

1. Manejadores locales: métodos anotados con @ExceptionHandler dentro del propio controlador.
2. Manejadores globales: métodos anotados con @ExceptionHandler dentro de una clase anotada con @RestControllerAdvice.

Si hay un manejador local que coincida, se usa ese. Si no, se busca en los globales. Si tampoco hay, Spring Boot devuelve su respuesta de error por defecto (la Whitelabel Error Page o el JSON de error estándar de Spring Boot).

La idea es centralizar el manejo de errores en una sola clase. En lugar de repetir try-catch o @ExceptionHandler en cada controlador, se define un manejador global que se aplica a todos.

T1.2 - @ControllerAdvice vs @RestControllerAdvice

Spring ofrece dos anotaciones para el manejo global:

@ControllerAdvice: la original. Los métodos pueden devolver el nombre de una vista o un objeto. Si devuelven un objeto, hay que anotar el método con @ResponseBody para que se serialice a JSON. Se usa en aplicaciones web tradicionales con vistas.

@RestControllerAdvice: especialización de @ControllerAdvice que añade @ResponseBody a todos los métodos. Los métodos devuelven directamente el cuerpo de la respuesta. Se usa en APIs REST.

La diferencia es la misma que entre @Controller y @RestController: en el primero, hay que anotar cada método con @ResponseBody; en el segundo, se aplica a todos por defecto. En este curso, y para cualquier API REST, se usa @RestControllerAdvice.

T1.3 - Cómo funciona internamente

Cuando Spring Boot arranca, escanea el paquete raíz buscando clases anotadas con @ControllerAdvice o @RestControllerAdvice. Para cada una:

1. Crea un bean y lo registra en el contexto.
2. Escanea sus métodos buscando @ExceptionHandler.
3. Registra cada método como manejador de la excepción indicada en la anotación.
4. Crea unExceptionHandlerExceptionResolver que consulta esa lista cuando un controlador lanza una excepción.

Cuando un controlador lanza una excepción:

1. El DispatcherServlet captura la excepción.
2. ElExceptionHandlerExceptionResolver busca un manejador que coincida con el tipo de excepción.
3. Si encuentra uno en el controlador, lo usa.
4. Si no, busca en los @RestControllerAdvice.
5. Si encuentra uno, invoca el método del manejador con la excepción como argumento.
6. El método construye una respuesta y la devuelve.
7. El DispatcherServlet envía esa respuesta al cliente.

El orden de búsqueda en los `@RestControllerAdvice` depende de `@Order`. Si dos manejadores manejan la misma excepción, se usa el de mayor prioridad (menor número en `@Order`).

Pregunta de reflexión: ¿Por qué el manejador local tiene prioridad sobre el global?
¿Qué ventaja tiene eso?

Bloque 2 - Estructura del manejador global

T2.1 - Un manejador por tipo de excepción

Un `@RestControllerAdvice` típico tiene un método por cada tipo de excepción que quiere manejar:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RecursoNoEncontradoException.class)
    public ResponseEntity<ErrorResponse> handleNoEncontrado(
        RecursoNoEncontradoException ex, HttpServletRequest request) {
        // ...
    }

    @ExceptionHandler(RecursoDuplicadoException.class)
    public ResponseEntity<ErrorResponse> handleDuplicado(
        RecursoDuplicadoException ex, HttpServletRequest request) {
        // ...
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGenerico(
        Exception ex, HttpServletRequest request) {
        // ...
    }
}
```

Cada método:

- Está anotado con `@ExceptionHandler` y recibe la clase de la excepción.
- Recibe la excepción como primer parámetro.
- Puede recibir `HttpServletRequest` para acceder a la URL, cabeceras, etc.
- Devuelve `ResponseEntity` (o cualquier objeto que se serialice a JSON).
- Construye la respuesta con el código HTTP y el cuerpo adecuados. El orden de los métodos en la clase no importa. Spring los registra todos y busca el más específico cuando se lanza una excepción.

T2.2 - Jerarquía de excepciones y especificidad

Cuando hay una jerarquía de excepciones, Spring busca el manejador más específico. Si lanzas `RecursoNoEncontradoException`, y tienes manejadores para `RecursoNoEncontradoException` y para `AplicacionException` (su clase base), se usa el de `RecursoNoEncontradoException`. Si solo tienes el de `AplicacionException`, se usa ese.

Ejemplo:

```
@ExceptionHandler(RecursoNoEncontradoException.class)
public ResponseEntity<ErrorResponse> handleNoEncontrado(...) { ... }

@ExceptionHandler(AplicacionException.class)
public ResponseEntity<ErrorResponse> handleAplicacion(...) { ... }
```

```
@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleGenerico(...) { ... }
```

Si se lanza `RecursoNoEncontradoException`, se usa el primer manejador. Si se lanza `OperacionNoPermitidaException` (que extiende `AplicacionException` pero no tiene manejador específico), se usa el segundo. Si se lanza `NullPointerException`, se usa el tercero.

Esta jerarquía permite tener un manejador genérico que capture todo lo que no esté específicamente manejado, garantizando que ningún error llegue al cliente sin estructura.

T2.3 - Orden con @Order

Cuando hay varios `@RestControllerAdvice` en la aplicación, se puede controlar el orden con `@Order`. El de menor valor tiene mayor prioridad.

```
@RestControllerAdvice
@Order(1)
public class ValidationExceptionHandler {
    // Maneja excepciones de validación
}
```

```
@RestControllerAdvice
@Order(2)
public class BusinessExceptionHandler {
    // Maneja excepciones de negocio
}
```

```
@RestControllerAdvice
@Order(3)
public class GenericExceptionHandler {
    // Maneja excepciones genéricas
}
```

Si se lanza `MethodArgumentNotValidException`, se busca primero en `ValidationExceptionHandler` (orden 1). Si no está, se busca en `BusinessExceptionHandler` (orden 2), y así sucesivamente.

En la práctica, la mayoría de los proyectos tienen un solo `@RestControllerAdvice`. Separar en varios solo tiene sentido cuando la lógica es muy distinta o cuando hay módulos independientes.

Pregunta de reflexión: ¿Qué ventaja tiene separar los manejadores en varios `@RestControllerAdvice` con `@Order`? ¿Cuándo lo harías?

Bloque 3 - Aplicación selectiva

T3.1 - Aplicar a paquetes concretos

Por defecto, `@RestControllerAdvice` se aplica a todos los controladores de la aplicación. Pero se puede restringir a paquetes, clases - anotaciones concretas.

Por paquete:

```
@RestControllerAdvice(basePackages = "es.mecd.demo.miproyecto.alumno")
public class AlumnoExceptionHandler {
    // Solo se aplica a los controladores del paquete alumno
}
```

Por clase:

```
@RestControllerAdvice(
    assignableTypes = {
        AlumnoController.class,
        CursoController.class
    })
```

```

    })
    public class SpecificExceptionHandler {
        // Solo se aplica a esos controladores
    }

```

Por anotación:

```

@RestControllerAdvice(annotations = RestController.class)
public class AllRestControllerHandler {
    // Se aplica a todos los controladores anotados con @RestController
}

```

En la práctica, la mayoría de los proyectos usan `@RestControllerAdvice` sin restricciones: un solo manejador global para toda la aplicación. Las restricciones se usan cuando hay módulos muy independientes que necesitan manejo distinto.

T3.2 - Combinación de manejadores locales y globales

Un controlador puede tener sus propios `@ExceptionHandler` además del manejador global. Cuando se lanza una excepción:

1. Spring busca primero en el controlador.
2. Si encuentra un manejador local, lo usa.
3. Si no, busca en los globales.

Esto es útil cuando un controlador necesita un manejo específico que no aplica al resto. Por ejemplo, un controlador que maneja ficheros podría tener un manejador específico para `FileNotFoundException` que devuelva un mensaje distinto.

```

@RestController
@RequestMapping("/api/v1/ficheros")
public class FicheroController {
    @ExceptionHandler({FileNotFoundException.class})
    public ResponseEntity<ErrorResponse> handleFileNotFoundException(
        FileNotFoundException ex, HttpServletRequest request) {
        // Manejo específico para este controlador
    }

    // ... endpoints
}

```

El resto de excepciones las maneja el `@RestControllerAdvice` global.

Pregunta de reflexión: ¿Cuándo usarías un manejador local en lugar de añadirlo al global?

T3.3 - Prioridad y orden de búsqueda

El orden de búsqueda completo es:

1. Manejador local en el controlador que lanza la excepción.
2. Manejador en un `@RestControllerAdvice` aplicable (por paquete, clase o anotación), ordenado por `@Order`.
3. Manejador en un `@RestControllerAdvice` global, ordenado por `@Order`.
4. `ResponseEntityExceptionHandler` de Spring Boot (si el manejador extiende esa clase), que maneja excepciones estándar de Spring MVC.
5. Respuesta por defecto de Spring Boot (Whitelabel Error Page o JSON de error estándar).

Conocer este orden ayuda a diagnosticar por qué un error se maneja de una forma y no de otra. Si un manejador local captura una excepción que querías que manejara el global, es porque el local tiene prioridad.

Pregunta de reflexión: ¿Qué pasa si un controlador tiene un `@ExceptionHandler` para `Exception` y también hay un `@RestControllerAdvice` con un manejador para `RecursoNoEncontradoException`? ¿Cuál se usa?

Bloque 4 - Casos especiales

T4.1 - Manejo de excepciones de validación

Las excepciones de validación son las más comunes en una API REST. Ya las hemos visto en el punto 5.2, pero conviene recordar cómo se manejan globalmente.

`MethodArgumentNotValidException`: se lanza cuando `@Valid` falla en el cuerpo de la petición. Contiene un `BindingResult` con los `FieldError`. `ConstraintViolationException`: se lanza cuando la validación falla en parámetros de método (`@RequestParam`, `@PathVariable`). Contiene un `Set<ConstraintViolation<?>`.

`HttpMessageNotReadableException`: se lanza cuando el cuerpo no se puede leer (JSON mal formado, `Content-Type` incorrecto).

`MethodArgumentTypeMismatchException`: se lanza cuando un parámetro no se puede convertir al tipo esperado.

`MissingServletRequestParameterException`: se lanza cuando falta un `@RequestParam` obligatorio.

Cada una de estas excepciones tiene su propio manejador en el `@RestControllerAdvice`. Todos devuelven 400 con el código y los detalles correspondientes.

T4.2 - Manejo de excepciones de seguridad

Cuando se usa Spring Security (Módulo 6), hay excepciones específicas que se lanzan durante la autenticación y autorización:

`AuthenticationException`: el cliente no se ha autenticado. Se traduce a 401.

`AccessDeniedException`: el cliente está autenticado pero no tiene permisos. Se traduce a 403. Estas excepciones no se manejan con `@RestControllerAdvice`. Spring Security las maneja en sus propios filtros, antes de que lleguen al `DispatcherServlet`. Para personalizar la respuesta, se configuran manejadores específicos en la configuración de seguridad (`AuthenticationEntryPoint` y `AccessDeniedHandler`).

Lo veremos en el Módulo 6.

T4.3 - Manejo de excepciones técnicas

Las excepciones técnicas son las que no tienen que ver con la lógica de negocio:

`DataAccessException`: fallo al acceder a la base de datos. Spring la lanza cuando Hibernate no puede ejecutar una consulta.

`HttpClientErrorException` y `HttpServerErrorException`: fallo al llamar a un servicio externo con `RestTemplate`.

`TimeoutException`: una operación ha superado el tiempo máximo.

`IOException`: fallo al leer o escribir un fichero. Todas estas excepciones se pueden manejar en el `@RestControllerAdvice` con un manejador genérico que devuelva 500 y registre el error en los logs. Nunca se debe exponer el mensaje original al cliente, porque puede contener información sensible (nombres de tablas, rutas de ficheros, etc.).

```
@ExceptionHandler(DataAccessException.class)
public ResponseEntity<ErrorResponse> handleDataAccess(
    DataAccessException ex, HttpServletRequest request) {
    log.error("Error de acceso a datos en {}", request.getRequestURI(), ex);
```

```

    return buildErrorResponse(
        HttpStatus.INTERNAL_SERVER_ERROR,
        "ERROR_BASE_DATOS",
        "Error al acceder a los datos",
        request,
        null
    );
}

```

El `log.error` registra la excepción completa (con traza de pila) en los logs. El cliente recibe un mensaje genérico.

Pregunta de reflexión: ¿Por qué no se debe exponer el mensaje original de una `DataAccessException` al cliente?

Bloque 5 - Buenas prácticas y tests

T5.1 - Buenas prácticas con `@RestControllerAdvice`

Un manejador global por aplicación. Salvo que haya módulos independientes, un solo `@RestControllerAdvice` es suficiente.

Un método por tipo de excepción. Cada excepción tiene su manejador. Eso permite devolver códigos y mensajes distintos.

Un manejador genérico para `Exception`. Captura todo lo que no esté específicamente manejado. Garantiza que ningún error llegue sin estructura.

Registrar los errores en los logs. Especialmente los técnicos. Usar `log.error` con la excepción completa.

No exponer detalles técnicos al cliente. El mensaje debe ser claro para el usuario, no para el desarrollador. Los detalles van a los logs.

Usar un método de ayuda para construir la respuesta. Evita repetir código.

Incluir un `traceId`. Permite correlacionar la respuesta del cliente con el log del servidor.

Documentar los códigos de error. En OpenAPI, listar los códigos posibles por endpoint.

Testear los manejadores. Verificar que devuelven el código y la estructura correctos. ### T5.2 - Tests del manejador global

Los tests del manejador global se hacen con `MockMvc`. Se simula una petición que provoca una excepción y se verifica la respuesta:

```

@WebMvcTest(AlumnoController.class)
class AlumnoControllerExceptionTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private AlumnoService service;

    @Test
    void consultar_debeDevolver404ConEstructuraEstandar() throws Exception {
        when(service.consultar(999L))
            .thenReturn(new RecursoNoEncontradoException("Alumno", 999L));

        mockMvc.perform(get("/api/v1/alumnos/999"))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath("$.codigo").value("RECURSO_NO_ENCONTRADO"));
    }
}

```

```

        .andExpect(jsonPath("$.detalles.recurso").value("Alumno"))
        .andExpect(jsonPath("$.detalles.id").value(999));
    }

@Test
void crear_debeDevolver409ConEstructuraEstandar() throws Exception {
    when(service.crear(any()))
        .thenThrow(new RecursoDuplicadoException("Alumno", "dni", "12345678A"));

    mockMvc.perform(post("/api/v1/alumnos")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
        {"nombre": "Ana", "apellidos": "García", "dni": "12345678A",
        "fechaNacimiento": "2010-05-12", "curso": "5º Primaria"}
        """))
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.codigo").value("RECURSO_DUPLICADO"));
}
}

```

El test verifica que el manejador global captura la excepción y devuelve la respuesta esperada.

T5.3 - Tests para excepciones no controladas

También conviene testear que el manejador genérico captura excepciones inesperadas:

```

@Test
void consultar_debeDevolver500ConEstructuraEstandar_
    cuandoErrorInesperado() throws Exception {
    when(service.consultar(anyLong()))
        .thenThrow(new RuntimeException("Error inesperado"));

    mockMvc.perform(get("/api/v1/alumnos/1"))
        .andExpect(status().isInternalServerError())
        .andExpect(jsonPath("$.codigo").value("ERROR_INTERNO"))
        .andExpect(jsonPath("$.mensaje").value("Error interno del servidor"));
}

```

El test verifica que una RuntimeException no controlada se traduce a 500 con un mensaje genérico. El mensaje original no se expone.

Pregunta de reflexión: ¿Por qué es importante testear el manejador genérico? ¿Qué error detecta?

Resumen de la teoría

- @RestControllerAdvice: manejo global de excepciones. Aplica a todos los controladores.
- @ControllerAdvice vs @RestControllerAdvice: el segundo añade @ResponseBody.
- Orden de búsqueda: local → global aplicable → global → ResponseEntityExceptionHandler → por defecto.
- Jerarquía de excepciones: Spring usa el manejador más específico.
- @Order: ordena los @RestControllerAdvice.
- Aplicación selectiva: por paquete (basePackages), clase (assignableTypes) o anotación (annotations).
- Excepciones de validación: MethodArgumentNotValidException, ConstraintViolationException, etc.
- Excepciones de seguridad: AuthenticationException y AccessDeniedException se manejan en Spring Security, no en @RestControllerAdvice.

- Excepciones técnicas: `DataAccessException`, `HttpClientErrorException`, etc.
- Buenas prácticas: un global, un método por excepción, manejador genérico, logs, no exponer detalles, `traceId`.
- Tests: con `MockMvc`, verificando código y estructura.

Punto 5.4 - Configuración básica de perfiles y properties

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar cómo Spring Boot carga la configuración desde `application.properties` y `application.yml`.
2. Diferenciar propiedades de Spring Boot de propiedades personalizadas.
3. Crear y activar perfiles para distintos entornos.
4. Usar `@Profile` para activar beans según el entorno.
5. Externalizar la configuración con `@ConfigurationProperties`.
6. Entender el orden de prioridad de las propiedades.
7. Diagnosticar y resolver los errores más comunes al configurar perfiles.

Bloque 1 - Configuración externalizada

T1.1 - Qué es `application.properties` y cómo se carga

`application.properties` es el archivo de configuración principal de una aplicación Spring Boot. Está en `src/main/resources/` y contiene pares clave-valor que Spring Boot lee al arrancar.

Cuando la aplicación arranca, Spring Boot busca el archivo en varias ubicaciones por orden de prioridad:

1. `config/` en el directorio actual (útil para sobrescribir configuración al desplegar).
2. El directorio actual (donde se ejecuta el JAR).
3. `classpath:/config/` (dentro del JAR, en una carpeta `config`).
4. `classpath:/` (dentro del JAR, en la raíz).

La primera ubicación donde encuentra el archivo es la que se usa. Si hay varios, los de mayor prioridad sobrescriben a los de menor. Esta flexibilidad permite tener una configuración por defecto en el JAR y sobrescribirla al desplegar sin recompilar.

El archivo se lee una vez, al arrancar. Si lo modificas en caliente, no se aplica hasta que reinicias la aplicación (salvo que uses DevTools, que la recarga automáticamente).

T1.2 - `application.properties` vs `application.yml`

Spring Boot admite dos formatos de archivo de configuración:

`application.properties`: formato clave-valor. Cada propiedad en una línea, con `=` o `:` entre la clave y el valor. Es el formato más tradicional.

```
server.port=8080
spring.datasource.url=jdbc:h2:mem:testdb
app.nombre-oficina=Oficina de Becas
```

`application.yml`: formato YAML. Usa indentación para representar jerarquía. Es más conciso y más legible para configuraciones anidadas.

```
server:
  port: 8080
spring:
  datasource:
```

```
url: jdbc:h2:mem:testdb
app:
  nombre-oficina: Oficina de Becas
```

Ambos formatos son equivalentes. Si tienes los dos, `properties` tiene prioridad sobre `yml`. En la práctica, se elige uno y se mantiene durante todo el proyecto. El equipo debe decidir cuál usar y ser consistente.

Ventajas de `properties`:

- Sintaxis simple.
- Fácil de leer para configuraciones planas.
- Menos errores de indentación.

Ventajas de `yml`:

- Más conciso para configuraciones anidadas.
- Permite listas y mapas de forma natural.
- Menos repetición de prefijos.

Para este curso usaremos `application.properties` por simplicidad.

T1.3 - Propiedades de Spring Boot y propiedades personalizadas

Spring Boot tiene cientos de propiedades con valores por defecto. Se organizan por prefijos:

- `server.*`: configuración del servidor (puerto, contexto, SSL).
- `spring.*`: configuración de Spring (datasource, JPA, Jackson, seguridad).
- `logging.*`: configuración de logs (nivel, formato, archivo).
- `management.*`: configuración de Actuador.
- `springdoc.*`: configuración de Springdoc OpenAPI.

Estas propiedades tienen valores por defecto. Solo hay que declarar las que se quieren cambiar. Por ejemplo, `server.port` por defecto es 8080; si no se declara, se usa ese valor.

Además de las propiedades de Spring Boot, se pueden definir propiedades personalizadas con un prefijo propio:

```
app.nombre-oficina=Oficina de Becas
app.version=1.0.0
app.cors.allowed-origins=http://localhost:3000
```

Por convención, se usa un prefijo propio (`app.`, `miempresa.`, `demo.`) para evitar conflictos con las propiedades de Spring Boot. Estas propiedades se leen con `@Value` o con `@ConfigurationProperties`.

Pregunta de reflexión: ¿Por qué es recomendable usar un prefijo propio para las propiedades personalizadas?

Bloque 2 - Perfiles de Spring Boot

T2.1 - Qué son los perfiles y por qué existen

Un perfil es un conjunto de configuración que se aplica solo cuando el perfil está activo. Permiten tener configuraciones distintas para distintos entornos (desarrollo, test, producción) sin cambiar el código.

Sin perfiles, tendrías que tener un solo `application.properties` con toda la configuración. Para cambiar de entorno, tendrías que modificar el archivo y recompilar. Con perfiles, tienes un archivo por entorno y activas el que quieras.

Los entornos típicos son:

- `dev`: desarrollo local. Base de datos en memoria (H2), logs verbosos.

- test: tests automatizados. Base de datos en memoria, logs mínimos.
- prod: producción. Base de datos real (PostgreSQL), logs controlados.

Cada perfil tiene su propio archivo: `application-dev.properties`, `application-test.properties`, `application-prod.properties`.

T2.2 - `application-{perfil}.properties`

La convención de nombres es `application-{perfil}.properties`. Spring Boot carga:

1. `application.properties` siempre.
2. `application-{perfil}.properties` para cada perfil activo.

Las propiedades del archivo del perfil sobrescriben las del archivo base. Así se puede tener una configuración común en `application.properties` y sobrescribir solo lo que cambia en cada entorno.

Ejemplo:

`application.properties` (común):

```
spring.application.name=mi-proyecto
app.version=1.0.0
app.nombre-oficina=Oficina de Becas
```

`application-dev.properties`:

```
server.port=8080
spring.datasource.url=jdbc:h2:mem:testdb
spring.jpa.show-sql=true
logging.level.es.mecd.demo=DEBUG
app.nombre-oficina=Oficina de Becas (Desarrollo)
```

`application-prod.properties`:

```
server.port=8443
spring.datasource.url=jdbc:postgresql://localhost:5432/mibd
spring.jpa.show-sql=false
logging.level.es.mecd.demo=INFO
app.nombre-oficina=Oficina de Becas
```

Cuando el perfil dev está activo, `app.nombre-oficina` vale "Oficina de Becas (Desarrollo)". Cuando el perfil prod está activo, vale "Oficina de Becas" (el valor base, porque prod no lo sobrescribe).

T2.3 - Cómo activar perfiles

Hay varias formas de activar un perfil, en orden de prioridad:

1. Argumento de línea de comandos:

```
java -jar app.jar --spring.profiles.active=prod
```

2. Variable de entorno:

```
export SPRING_PROFILES_ACTIVE=prod
java -jar app.jar
```

3. Propiedad en `application.properties`:

```
spring.profiles.active=dev
```

Esta es la forma más común en desarrollo. En producción, se usa el argumento de línea de comandos o la variable de entorno, para no tener que modificar el JAR.

4. Variable de entorno en el IDE:

En IntelliJ, Eclipse o VS Code, se puede configurar la variable `SPRING_PROFILES_ACTIVE` en la configuración de ejecución.

Si no se activa ningún perfil, Spring Boot usa el perfil default. Si no hay `application-default.properties`, se usa solo `application.properties`.

Se pueden activar varios perfiles a la vez:

```
java -jar app.jar --spring.profiles.active=dev,metrics
```

Los perfiles se aplican en orden. Si dos perfiles definen la misma propiedad, gana el último. Esto permite tener perfiles “transversales” (por ejemplo, `metrics` para activar métricas) que se combinan con el perfil de entorno.

Pregunta de reflexión: ¿Por qué es mala idea tener `spring.profiles.active=prod` en `application.properties` cuando se despliega en producción?

Bloque 3 - @Profile y beans condicionales

T3.1 - @Profile en clases @Component/@Service

La anotación `@Profile` permite que un bean solo se registre cuando un perfil concreto está activo. Se puede poner en cualquier clase anotada con `@Component`, `@Service`, `@Repository`, `@Configuration`, etc.

```
@Service
@Profile("dev")
public class DatosInicialesDevService {
    // Este servicio solo se registra cuando el perfil dev está activo
}
```

Si el perfil `dev` no está activo, Spring no crea el bean. Si algún otro bean lo inyecta, Spring falla al arrancar con `NoSuchBeanDefinitionException`. Por eso hay que asegurarse de que las dependencias estén alineadas con los perfiles.

Se pueden usar expresiones más complejas:

```
@Profile({"dev", "test"})    // Se registra en dev o test
@Profile("!prod")           // Se registra en cualquier perfil excepto prod
@Profile("dev & metrics")    // Se registra solo si ambos están activos
@Profile("dev | test")       // Se registra si cualquiera está activo
```

Estas expresiones permiten definir condiciones precisas para cada bean.

T3.2 - @Profile en métodos @Bean

`@Profile` también se puede poner en métodos anotados con `@Bean` dentro de una clase `@Configuration`:

```
@Configuration
public class DatosInicialesConfig {

    @Bean
    @Profile("dev")
    public CommandLineRunner cargarDatosDev(AlumnoRepository repositorio) {
        return args -> {
            // Cargar datos de ejemplo solo en desarrollo
        };
    }

    @Bean
    @Profile("prod")
    public CommandLineRunner verificarDatosProd(AlumnoRepository repositorio) {
```

```

    return args -> {
        // Verificar que hay datos en producción
    };
}
}

```

Los dos beans CommandLineRunner se registran solo cuando el perfil correspondiente está activo. En dev se cargan datos de ejemplo;

en prod se verifica que hay datos. En otros perfiles, ninguno se ejecuta.

Esta técnica es muy útil para tener datos de ejemplo solo en desarrollo y verificaciones solo en producción.

T3.3 - Beans por defecto y beans específicos

A veces se quiere tener un bean por defecto y otro que lo sustituya en un perfil concreto. Se hace con @Profile y @Primary:

```

public interface NotificacionService {
    void enviar(String destinatario, String mensaje);
}
@Service
@Profile("!prod")
public class NotificacionServiceMock implements NotificacionService {
    @Override
    public void enviar(String destinatario, String mensaje) {
        System.out.println("MOCK: " + destinatario + " - " + mensaje);
    }
}

@Service
@Profile("prod")
@Primary
public class NotificacionServiceReal implements NotificacionService {
    @Override
    public void enviar(String destinatario, String mensaje) {
        // Enviar email real
    }
}

```

En desarrollo y test se usa el mock, que solo imprime por consola. En producción se usa el real, que envía emails. El @Primary indica que, si hay varios beans del mismo tipo, este tiene prioridad.

Pregunta de reflexión: ¿Por qué es útil tener un servicio mock en desarrollo y uno real en producción?

Bloque 4 - @ConfigurationProperties

T4.1 - Qué es @ConfigurationProperties

@ConfigurationProperties es una anotación que permite mapear un grupo de propiedades a una clase Java. En lugar de inyectar cada propiedad con @Value, se inyecta un objeto con todos los campos.

Sin @ConfigurationProperties:

```

@Component
public class InfoApp {

    @Value("${app.nombre-oficina}")

```

```

    private String nombreOficina;

    @Value("${app.version}")
    private String version;

    @Value("${app.entorno}")
    private String entorno;
}

```

Con @ConfigurationProperties:

```

@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {

    private String nombreOficina;
    private String version;
    private String entorno;

    // getters y setters
}

```

prefix = "app" indica que esta clase mapea las propiedades que empiezan por app.. El campo nombreOficina se mapea a app.nombre-oficina (Spring convierte camelCase a kebab-case automáticamente).

Las ventajas de @ConfigurationProperties:

- Agrupa propiedades relacionadas en una sola clase.
- Permite validación con Bean Validation (@NotBlank, @NotNull, etc.).
- Permite valores por defecto en los campos.
- Es más tipado que @Value.
- Facilita el testing de la configuración.

T4.2 - Estructuras anidadas

@ConfigurationProperties permite mapear estructuras anidadas, listas y mapas:

```

app.nombre-oficina=Oficina de Becas
app.version=1.0.0
app.cors.allowed-origins=http://localhost:3000,http://localhost:4200
app.cors.allowed-methods=GET,POST,PUT,DELETE
app.notificaciones.email.host=smtp.educacion.gob.es
app.notificaciones.email.puerto=587
app.notificaciones.sms.habilitado=false

```

La clase correspondiente:

```

@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {

    private String nombreOficina;
    private String version;
    private Cors cors = new Cors();
    private Notificaciones notificaciones = new Notificaciones();

    public static class Cors {
        private List<String> allowedOrigins = new ArrayList<>();
        private List<String> allowedMethods = new ArrayList<>();
    }
}

```

```

        // getters y setters
    }
    public static class Notificaciones {
        private Email email = new Email();
        private Sms sms = new Sms();
        // getters y setters
    }

    public static class Email {
        private String host;
        private int puerto;
        // getters y setters
    }

    public static class Sms {
        private boolean habilitado;
        // getters y setters
    }

    // getters y setters
}

```

Spring mapea las propiedades anidadas a las clases internas. Las listas se convierten de String separado por comas a List. ### T4.3 - Validación de propiedades

@ConfigurationProperties se puede combinar con Bean Validation para validar que las propiedades tienen los valores correctos:

```

@Component
@ConfigurationProperties(prefix = "app")
@Validated
public class AppProperties {

    @NotBlank
    private String nombreOficina;

    @NotBlank
    @Pattern(regexp = "\\d+\\.\\d+\\.\\d+")
    private String version;

    @NotNull
    private String entorno;

    // getters y setters
}

```

Nota sobre @Validated. Las anotaciones @NotBlank y @Pattern son las mismas que usamos en los DTOs de entrada (Módulo 3). Pero

aquí el contexto es distinto: no validan datos enviados por un usuario, sino propiedades de configuración definidas en application.properties. @Validated activa la validación al arrancar la aplicación. Si alguna propiedad no cumple las restricciones (por ejemplo, app.version no tiene formato x.y.z), Spring Boot lanza una excepción y la aplicación no arranca. Es preferible que falle al arrancar a que falle en tiempo de ejecución.

Esta validación es útil para detectar errores de configuración rápidamente. Si alguien despliega en producción sin definir app.nombre-oficina, la aplicación falla al arrancar con un mensaje claro.

Pregunta de reflexión: ¿Qué ventaja tiene que la aplicación falle al arrancar si falta una propiedad obligatoria?

Bloque 5 - Orden de prioridad y buenas prácticas

T5.1 - Orden de prioridad de las propiedades

Spring Boot carga las propiedades de muchas fuentes, y algunas tienen prioridad sobre otras. El orden completo (de mayor a menor prioridad) es:

1. Argumentos de línea de comandos (`-server.port=9090`).
2. Variables de entorno (`SERVER_PORT=9090`).
3. `application-{perfil}.properties` fuera del JAR.
4. `application-{perfil}.properties` dentro del JAR.
5. `application.properties` fuera del JAR.
6. `application.properties` dentro del JAR.
7. `@PropertySource` en clases `@Configuration`.
8. Valores por defecto de Spring Boot. Esta jerarquía permite que la configuración más específica sobrescriba a la más general. Por ejemplo, se puede tener un `application.properties` con valores por defecto y sobrescribir el puerto con un argumento de línea de comandos al desplegar.

En la práctica:

- Desarrollo: `application.properties` + `application-dev.properties`.
- Test: `application.properties` + `application-test.properties`.
- Producción: `application.properties` + `application-prod.properties` + variables de entorno para secretos.

T5.2 - Secretos y variables de entorno

Los secretos (contraseñas de base de datos, claves de API, tokens JWT) nunca deben escribirse en `application.properties`. El archivo va al repositorio y cualquiera con acceso puede verlo.

La forma correcta es usar variables de entorno:

```
spring.datasource.password=${DB_PASSWORD}
jwt.secret=${JWT_SECRET}
```

Y definir las variables en el sistema:

```
export DB_PASSWORD=contraseña_segura
export JWT_SECRET=clave_secreta_larga
```

Spring Boot lee las variables de entorno automáticamente. Si la propiedad no se encuentra en `application.properties`, busca en las variables de entorno. Si tampoco, lanza un error al arrancar (o usa el valor por defecto si se ha definido con `${VAR:default}`).

En desarrollo, se pueden usar archivos `.env` que no se suben al repositorio. La librería `spring-dotenv` los carga automáticamente. O simplemente se exportan las variables antes de arrancar la aplicación.

Otra opción para producción: Spring Cloud Config, un servidor de configuración centralizado que sirve las propiedades a las aplicaciones. Se verá en el Módulo 7.

T5.3 - Buenas prácticas con perfiles y properties

Usar perfiles para entornos. dev, test, prod. No mezclar configuraciones.

Un archivo por perfil. `application-dev.properties`, `application-prod.properties`.

Propiedades comunes en `application.properties`. Solo lo que es igual en todos los entornos.

Propiedades específicas en el archivo del perfil. Lo que cambia entre entornos.

Secretos en variables de entorno. Nunca en archivos que van al repositorio.

Usar `@ConfigurationProperties` para grupos de propiedades. Más limpio que múltiples `@Value`.

Validar las propiedades al arrancar. Con `@Validated` y Bean Validation. Documentar las propiedades. En el README, listar las propiedades que se pueden configurar.

Valores por defecto sensatos. Si una propiedad no se define, que la aplicación funcione.

No usar `spring.profiles.active` en `application.properties`. Mejor pasarlo por línea de comandos o variable de entorno.

Consistencia entre entornos. Si dev tiene 20 propiedades, prod debería tener las mismas (o más), no menos.

Pregunta de reflexión: ¿Por qué es mala idea tener `spring.profiles.active=dev` en `application.properties`?

Resumen de la teoría

- `application.properties`: archivo de configuración principal. Se busca en varias ubicaciones.
- `application.yml`: alternativa YAML. Equivalente a `properties`.
- Propiedades de Spring Boot: `server.`, `spring.`, `logging.`, `management.`.
- Propiedades personalizadas: con prefijo propio (`app.*`).
- Perfiles: `application-{perfil}.properties`. Se activan con `spring.profiles.active`.
- `@Profile`: activa beans según el perfil.
- `@ConfigurationProperties`: mapea grupos de propiedades a clases Java.
- Validación: `@Validated` + Bean Validation.
- Prioridad: argumentos > variables de entorno > perfil > base.
- Secretos: variables de entorno, nunca en archivos.
- Buenas prácticas: un archivo por perfil, secretos fuera, `@ConfigurationProperties`, validación.

Punto 5.5 - Configuración de CORS para consumo desde front-end

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar qué es la política de mismo origen (Same-Origin Policy) y por qué existe.
2. Describir qué es CORS y cómo funciona la negociación entre navegador y servidor.
3. Diferenciar peticiones simples de peticiones preflight (OPTIONS).
4. Configurar CORS globalmente con `WebMvcConfigurer`.
5. Configurar CORS a nivel de controlador con `@CrossOrigin`.
6. Usar perfiles para tener configuraciones de CORS distintas según el entorno.
7. Diagnosticar y resolver los errores más comunes al configurar CORS.

Bloque 1 - La política de mismo origen

T1.1 - Qué es el origen de una página web

Cuando un navegador carga una página web, esa página tiene un origen concreto. Un origen se define por la combinación de tres elementos:

- Esquema: `http` o `https`.
- Host: el dominio o la IP (`localhost`, `sede.educacion.gob.es`, `192.168.1.10`).
- Puerto: `80`, `443`, `8080`, `3000`.

Dos URLs tienen el mismo origen si los tres elementos coinciden. Si cambia cualquiera de los tres, el origen es distinto.

Ejemplos:

- <https://sede.educacion.gob.es/index.html> y <https://sede.educacion.gob.es/api/becas> → mismo origen.
- <http://sede.educacion.gob.es> y <https://sede.educacion.gob.es> → distinto origen (esquema distinto).
- <https://sede.educacion.gob.es> y <https://api.educacion.gob.es> → distinto origen (host distinto).
- <http://localhost:3000> y <http://localhost:8080> → distinto origen (puerto distinto).

Esta definición es clave para entender por qué el navegador bloquea algunas peticiones. ### T1.2 - La política de mismo origen

La Same-Origin Policy (SOP) es una política de seguridad que aplican todos los navegadores modernos. Dice: una página web solo puede hacer peticiones a su mismo origen. Si intenta hacer una petición a otro origen, el navegador la bloquea.

¿Por qué existe esta política? Para proteger al usuario. Imagina que estás logueado en tu banco en una pestaña. Abres otra pestaña con una web maliciosa. Sin la SOP, esa web podría hacer peticiones a la web del banco usando tus cookies y, por ejemplo, transferir dinero sin que te enteraras. La SOP evita eso: la web maliciosa no puede hacer peticiones al banco porque son orígenes distintos.

En el contexto de una API REST:

- El front-end se sirve desde un origen (por ejemplo, <http://localhost:3000> si es React en desarrollo).
- El back-end está en otro origen (por ejemplo, <http://localhost:8080>).
- Cuando el front-end intenta llamar al back-end, el navegador detecta que son orígenes distintos y bloquea la petición.

Es un error muy común en desarrollo: el back-end funciona perfectamente con curl o Postman (que no aplican SOP), pero el front-end no puede llamarlo. El navegador muestra un error como:

```
Access to XMLHttpRequest at
'http://localhost:8080/api/v1/alumnos' from origin
'http://localhost:3000' has been blocked by CORS policy:
No 'Access-Control-Allow-Origin' header is present on the requested resource.
```

Ese error es la razón por la que existe CORS. ### T1.3 - Qué es CORS

CORS son las siglas de Cross-Origin Resource Sharing (Compartición de Recursos entre Orígenes). Es un mecanismo que permite al servidor autorizar al navegador a hacer peticiones desde otros orígenes.

Es importante entender que CORS es una decisión del servidor, no del cliente. El navegador bloquea la petición, pero es el servidor quien, mediante cabeceras HTTP, le dice al navegador: “puedes dejar pasar esta petición”. Si el servidor no envía esas cabeceras, el navegador la bloquea.

El mecanismo funciona así:

1. El navegador detecta que el front-end (localhost:3000) quiere hacer una petición al back-end (localhost:8080), que es otro origen.
2. El navegador añade automáticamente la cabecera Origin: <http://localhost:3000> a la petición.
3. El servidor recibe la petición, comprueba si ese origen está permitido, y si lo está, responde con una cabecera Access-Control-Allow-Origin: <http://localhost:3000>.
4. El navegador ve esa cabecera y permite que la respuesta llegue al front-end.
5. Si el servidor no envía esa cabecera (o envía un origen distinto), el navegador bloquea la respuesta.

El servidor no bloquea la petición: la recibe y la procesa. Es el navegador quien bloquea la respuesta. Por eso, cuando depuras CORS, los logs del servidor muestran que la petición ha llegado, pero el front-end no recibe la respuesta.

Pregunta de reflexión: ¿Por qué curl y Postman no tienen problemas de CORS? ¿Qué diferencia hay con un navegador?

Bloque 2 - Peticiones simples y preflight

T2.1 - Peticiones simples

No todas las peticiones entre orígenes requieren el mismo tratamiento. Hay dos tipos: simples y preflight.

Una petición simple es la que cumple todas estas condiciones:

- Método: GET, HEAD o POST.
- Cabeceras: solo cabeceras simples (Accept, Accept-Language, Content-Language, Content-Type con valor limitado).
- Content-Type: solo application/x-www-form-urlencoded, multipart/form-data o text/plain.

Si la petición es simple, el navegador la envía directamente, sin preguntar al servidor. El servidor responde con las cabeceras CORS adecuadas, y el navegador decide si permite la respuesta.

Ejemplo: un GET a /api/v1/alumnos desde localhost:3000 con la cabecera Accept: application/json. Es simple.

T2.2 - Peticiones preflight

Una petición preflight es la que no cumple las condiciones de petición simple. Por ejemplo:

- Usa métodos PUT, PATCH o DELETE.
- Usa cabeceras personalizadas (por ejemplo, Authorization para JWT).
- Usa Content-Type: application/json. Cuando el navegador detecta una petición que no es simple, antes de enviarla, envía automáticamente una petición de tipo OPTIONS al mismo endpoint. Esa es la petición preflight.

El preflight lleva cabeceras que informan al servidor de qué quiere hacer la petición real:

- Origin: el origen del front-end.
- Access-Control-Request-Method: el método HTTP que va a usar la petición real (POST, PUT, DELETE...).
- Access-Control-Request-Headers: las cabeceras personalizadas que va a enviar la petición real (Authorization, Content-Type...).

El servidor responde al preflight con:

- Access-Control-Allow-Origin: los orígenes permitidos.
- Access-Control-Allow-Methods: los métodos permitidos.
- Access-Control-Allow-Headers: las cabeceras permitidas.
- Access-Control-Max-Age: cuánto tiempo puede el navegador cachear esa respuesta para no repetir el preflight.

Si el servidor responde correctamente al preflight, el navegador envía la petición real. Si el servidor no responde al preflight (o responde con cabeceras incorrectas), el navegador no envía la petición real.

Por eso, cuando configuras CORS, es crítico que el servidor maneje las peticiones OPTIONS. Si no las maneja, el preflight falla y la petición real nunca se envía.

T2.3 - Cuándo se produce el preflight

El preflight se produce cuando la petición cumple alguna de estas condiciones:

- Método no simple: PUT, PATCH, DELETE.
- Cabeceras personalizadas: cualquier cabecera que no sea simple (por ejemplo, Authorization, X-Request-Id).
- Content-Type no simple: application/json, application/xml, etc.

En una API REST moderna, casi todas las peticiones no-GET son preflight. Porque:

- Los POST con application/json no son simples (el Content-Type no lo es).
- Los PUT y DELETE no son simples (el método no lo es).
- Las peticiones con JWT llevan Authorization, que no es simple.

Por tanto, configurar CORS solo para GET no es suficiente: hay que configurarlo para todos los métodos y todas las cabeceras que use la aplicación.

Pregunta de reflexión: ¿Por qué un POST con application/json no es una petición simple?
¿Qué preflight se envía antes?

Bloque 3 - Cómo configurar CORS en Spring Boot

T3.1 - Configuración global con WebMvcConfigurer

La forma más limpia de configurar CORS en Spring Boot es con una clase @Configuration que implemente WebMvcConfigurer y sobrescriba el método addCorsMappings:

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:3000", "https://sede.educacion.gob.es")
            .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")
            .allowedHeaders("*")
            .exposedHeaders("Location")
            .allowCredentials(true)
            .maxAge(3600);
    }
}
```

Analicemos cada método:

- addMapping("/api/"): **los patrones de URL a los que se aplica esta configuración. /api/** cubre todos los endpoints bajo /api/.
- allowedOrigins(...): los orígenes permitidos. Solo las peticiones que vengan de estos orígenes serán autorizadas.
- allowedMethods(...): los métodos HTTP permitidos. Siempre incluir OPTIONS, porque es el método del preflight.
- allowedHeaders("*"): *las cabeceras permitidas en la petición real.* significa todas.
- exposedHeaders("Location"): las cabeceras de la respuesta que el front-end puede leer. Por defecto, el front-end solo puede leer unas pocas cabeceras estándar. Si quieres que pueda leer Location (útil en POST), hay que exponerla.
- allowCredentials(true): si se permiten cookies o cabeceras Authorization. Si se pone a true, allowedOrigins no puede ser *: hay que especificar los orígenes.
- maxAge(3600): cuántos segundos puede el navegador cachear la respuesta del preflight. Reduce el número de peticiones OPTIONS.

Esta configuración se aplica a todos los controladores que coincidan con el patrón de URL.

T3.2 - Configuración a nivel de controlador

Además de la configuración global, Spring permite configurar CORS a nivel de controlador o de método con la anotación @CrossOrigin:

```
@RestController
@RequestMapping("/api/v1/alumnos")
@CrossOrigin(origins = "http://localhost:3000")
public class AlumnoController {
```

```
// ...
}
```

O a nivel de método:

```
@CrossOrigin(origins = "http://localhost:3000")
@GetMapping("/{id}")
public ResponseEntity<AlumnoDTO> consultar(@PathVariable Long id) {
    // ...
}
```

Los atributos de @CrossOrigin son similares a los de CorsRegistry:

- origins: los orígenes permitidos.
- methods: los métodos permitidos.
- allowedHeaders: las cabeceras permitidas.
- exposedHeaders: las cabeceras expuestas.
- allowCredentials: si se permiten credenciales.
- maxAge: tiempo de caché del preflight.

Cuándo usar la configuración global vs la anotación:

- Global: cuando la mayoría de los endpoints comparten la misma configuración. Es lo más común.
- Anotación: cuando unos pocos endpoints necesitan una configuración distinta. Se combina con la global.

En un proyecto concreto, se suele usar la configuración global para todos los endpoints bajo /api/, y @CrossOrigin solo en casos específicos.

T3.3 - Nota sobre Spring Security

Cuando se usa Spring Security (lo veremos en el Módulo 6), la configuración de CORS debe integrarse con la cadena de filtros de seguridad. Si no se hace, Spring Security bloquea las peticiones preflight con un 401 o 403, y el navegador no puede consumir la API. Lo cubriremos en detalle en el Módulo 6.

Bloque 4 - Casos prácticos

T4.1 - Front-end en desarrollo

En desarrollo, el front-end se sirve normalmente desde un servidor local (React con npm start, Angular con ng serve, Vue con npm run dev). Ese servidor suele usar un puerto distinto al del back-end:

- Front-end: http://localhost:3000.
- Back-end: http://localhost:8080.

Son orígenes distintos. CORS es necesario.

La configuración típica en desarrollo:

```
registry.addMapping("/api/**")
.allowedOrigins(
    "http://localhost:3000",
    "http://localhost:4200",
    "http://localhost:5173")
.allowedMethods("*")
.allowedHeaders("*")
.allowCredentials(true)
.maxAge(3600);
```

Los puertos 3000, 4200 y 5173 son los que usan por defecto Create React App, Angular CLI y Vite, respectivamente. Se listan todos para que funcione con cualquier front-end.

T4.2 - Front-end en producción

En producción, el front-end y el back-end suelen servirse desde el mismo origen. Por ejemplo:

- Front-end: <https://sede.educacion.gob.es>.
- Back-end: <https://sede.educacion.gob.es/api/v1/...>

Como el origen es el mismo, no hay CORS. El navegador no bloquea las peticiones.

La forma de conseguirlo:

- Opción 1: un proxy inverso (Nginx, Apache) que sirve el front-end como estáticos y redirige `/api/**` al back-end. El cliente ve un solo origen.
- Opción 2: servir el front-end desde el propio Spring Boot (con Thymeleaf o como recursos estáticos en `src/main/resources/static/`). Pero es menos común en proyectos modernos.

En producción, la configuración de CORS puede ser más restrictiva:

```
registry.addMapping("/api/**")
.allowedOrigins("https://sede.educacion.gob.es")
.allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")
.allowedHeaders("Content-Type", "Authorization")
.exposedHeaders("Location")
.allowCredentials(true)
.maxAge(3600);
```

No usar `*` en producción. Especificar los orígenes concretos.

T4.3 - Diferencias entre desarrollo y producción

La configuración de CORS debería cambiar entre entornos. En desarrollo, es permisiva; en producción, restrictiva. Se usan perfiles (que ya conocemos del punto anterior):

application-dev.properties:

```
cors.allowed-origins=http://localhost:3000,http://localhost:4200
cors.allowed-methods=GET,POST,PUT,PATCH,DELETE,OPTIONS
```

application-prod.properties:

```
cors.allowed-origins=https://sede.educacion.gob.es
cors.allowed-methods=GET,POST,PUT,PATCH,DELETE,OPTIONS
```

Y en la clase CorsConfig se leen las propiedades:

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Value("${cors.allowed-origins}")
    private String[] allowedOrigins;

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins(allowedOrigins)
            .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")
            .allowedHeaders("")
            .allowCredentials(true)
            .maxAge(3600);
    }
}
```

```
}  
}
```

Ventaja: en desarrollo no hay que tocar código; en producción, la configuración se ajusta automáticamente con el perfil.

Pregunta de reflexión: ¿Por qué es mala idea dejar `allowedOrigins("*")` en producción?
¿Qué riesgo tiene?

Bloque 5 - Errores comunes y buenas prácticas

T5.1 - Errores comunes

Error 1: no incluir `OPTIONS` en `allowedMethods`.

El preflight se envía con el método `OPTIONS`. Si no está permitido, el servidor responde con 405 y el preflight falla. Siempre incluir `OPTIONS`.

Error 2: usar `allowedOrigins("*")` con `allowCredentials(true)`.

CORS no permite esta combinación. Si se permiten credenciales, hay que especificar orígenes concretos. El navegador rechazará la respuesta.

Error 3: no exponer la cabecera `Location`.

Si el front-end necesita leer la cabecera `Location` de un POST (para saber la URL del recurso creado), hay que exponerla con `exposedHeaders("Location")`. Por defecto, el front-end solo puede leer cabeceras simples (`Cache-Control`, `Content-Language`, `Content-Type`, `Expires`, `Last-Modified`, `Pragma`).

Error 4: configurar CORS solo en el controlador y no en el filtro de seguridad.

Si se usa Spring Security (Módulo 6), hay que configurar CORS en la cadena de filtros. La configuración de Spring MVC no llega.

Error 5: no configurar CORS para los errores. Si el back-end devuelve un 500 sin las cabeceras CORS, el front-end no puede leer el error. El `@RestControllerAdvice` y la configuración de CORS deben estar alineados.

Error 6: olvidar `allowCredentials(true)` cuando se usan cookies o `Authorization`.

Sin `allowCredentials(true)`, el navegador no envía las cookies ni la cabecera `Authorization`. La autenticación falla.

Error 7: usar `@CrossOrigin` sin incluir `OPTIONS`.

Igual que en el caso global. Si usas `@CrossOrigin` a nivel de controlador, asegúrate de que los métodos incluyen `OPTIONS` o de que Spring lo añade automáticamente.

Error 8: no cachear el preflight.

Sin `maxAge`, el navegador envía un preflight por cada petición. Eso multiplica las peticiones `OPTIONS`. Poner `maxAge(3600)` (1 hora).

Error 9: error "Response to preflight request doesn't pass access control check".

El servidor no está respondiendo al preflight. Verificar que `OPTIONS` está permitido y que Spring Security no lo bloquea.

Error 10: CORS funciona con curl pero no con el navegador.

curl no aplica SOP. El navegador sí. Los problemas de CORS solo se ven desde el navegador.

T5.2 - Buenas prácticas

Configurar CORS globalmente. Una sola clase CorsConfig para toda la aplicación.

Especificar orígenes concretos. No usar * en producción.

Incluir OPTIONS en los métodos permitidos.

Exponer las cabeceras necesarias. Location para POST, Authorization si se usa.

Usar allowCredentials(true) solo si se necesitan cookies o cabeceras de autenticación.

Cachear el preflight. maxAge(3600).

Cambiar la configuración por entorno. Con perfiles.

Probar CORS desde el navegador. No desde curl o Postman.

Documentar la configuración. En el README, indicar qué orígenes están permitidos.

Revisar la configuración cuando se añade un front-end nuevo.

T5.3 - Diagnóstico de errores CORS

Cuando algo falla con CORS, el primer paso es mirar la consola del navegador. El mensaje de error suele indicar qué cabecera falta:

- “No ‘Access-Control-Allow-Origin’ header is present” → el servidor no está enviando la cabecera. Verificar que la configuración de CORS se aplica al endpoint.
- “Response to preflight request doesn’t pass access control check” → el preflight falla. Verificar que OPTIONS está permitido y que Spring Security no lo bloquea.
- “The value of the ‘Access-Control-Allow-Origin’ header in the response must not be the wild-card ‘*’ when the request’s credentials mode is ‘include’” → se usa con allowCredentials(true). Especificar orígenes.
- “Request header field Authorization is not allowed by Access-Control-Allow-Headers” → falta la cabecera Authorization en allowedHeaders.

En el servidor, se puede activar el log DEBUG para ver qué peticiones OPTIONS llegan:

```
logging.level.org.springframework.web.cors=DEBUG
```

Con eso, se ven las peticiones que Spring procesa y la configuración que aplica.

Pregunta de reflexión: ¿Por qué los problemas de CORS solo se ven desde el navegador?
¿Qué herramienta usarías para diagnosticarlos?

Resumen de la teoría

- Origen: combinación de esquema, host y puerto.
- Same-Origin Policy: los navegadores bloquean peticiones entre orígenes distintos.
- CORS: mecanismo que permite al servidor autorizar al navegador.
- Petición simple: GET/HEAD/POST con cabeceras simples.
- Petición preflight: OPTIONS antes de la petición real. Se envía cuando la petición no es simple.
- Configuración global: WebMvcConfigurer.addCorsMappings.
- Configuración por controlador: @CrossOrigin.
- Con Spring Security: http.cors() y CorsConfigurationSource (Módulo 6).
- Buenas prácticas: OPTIONS permitido, orígenes concretos, maxAge, exponer cabeceras, perfiles.
- Errores comunes: no incluir OPTIONS, * con credenciales, no exponer Location, no configurar en seguridad.